Mini Appointment Booking System - System Design

Prepared By - Md. Sayeed Rahman

Requirements

We need to gather two types of requirements: -

- Functional Requirements
  - What the system should do
- Non-Functional Requirements
  - Performance, security, and usability

Functional Requirements

- Register clinics and doctors.

- A doctor can serve multiple clinics, but a slot cannot overlap across clinics.

- Doctors define available schedules (recurring time windows).

- Patients can search doctors or clinics and view available slots.

- Patients can book, cancel, or reschedule appointments.

- Each slot can be booked by only one patient.

- A patient cannot have overlapping appointments across clinics.

Non-Functional Requirements

- Support multiple clinics in a single backend (multi-tenant).

- Ensure clinic data is isolated and secure.

- Handle thousands of clinics and tens of thousands of concurrent users.

- Ensure strong consistency for booking operations.

- Slot search and availability queries should be fast (<200 ms).

- High availability to support clinic operations.

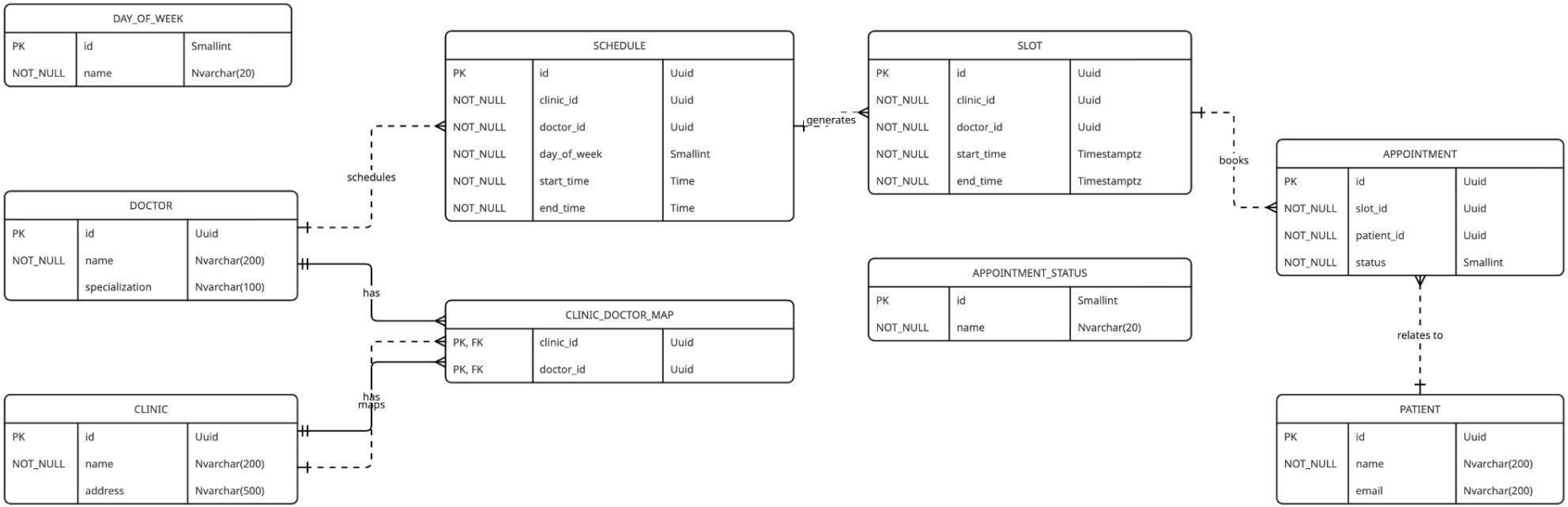
- System should be extensible for future features (notifications, online consultations, payments, analytics).

Entity Design & Data Modeling

Core Entities

Clinic – Represents a medical center or branch.
Doctor – Represents a healthcare provider who can work in multiple clinics.
ClinicDoctorMap – Manages the many-to-many relationship between clinics and doctors.
Schedule – Defines a doctor’s recurring availability (e.g., Monday 9 AM–5 PM).
Slot – Represents a bookable time interval generated from a schedule.
Patient – Represents a user of the system booking appointments.
Appointment – Represents the booking transaction and lifecycle.

ER Diagram

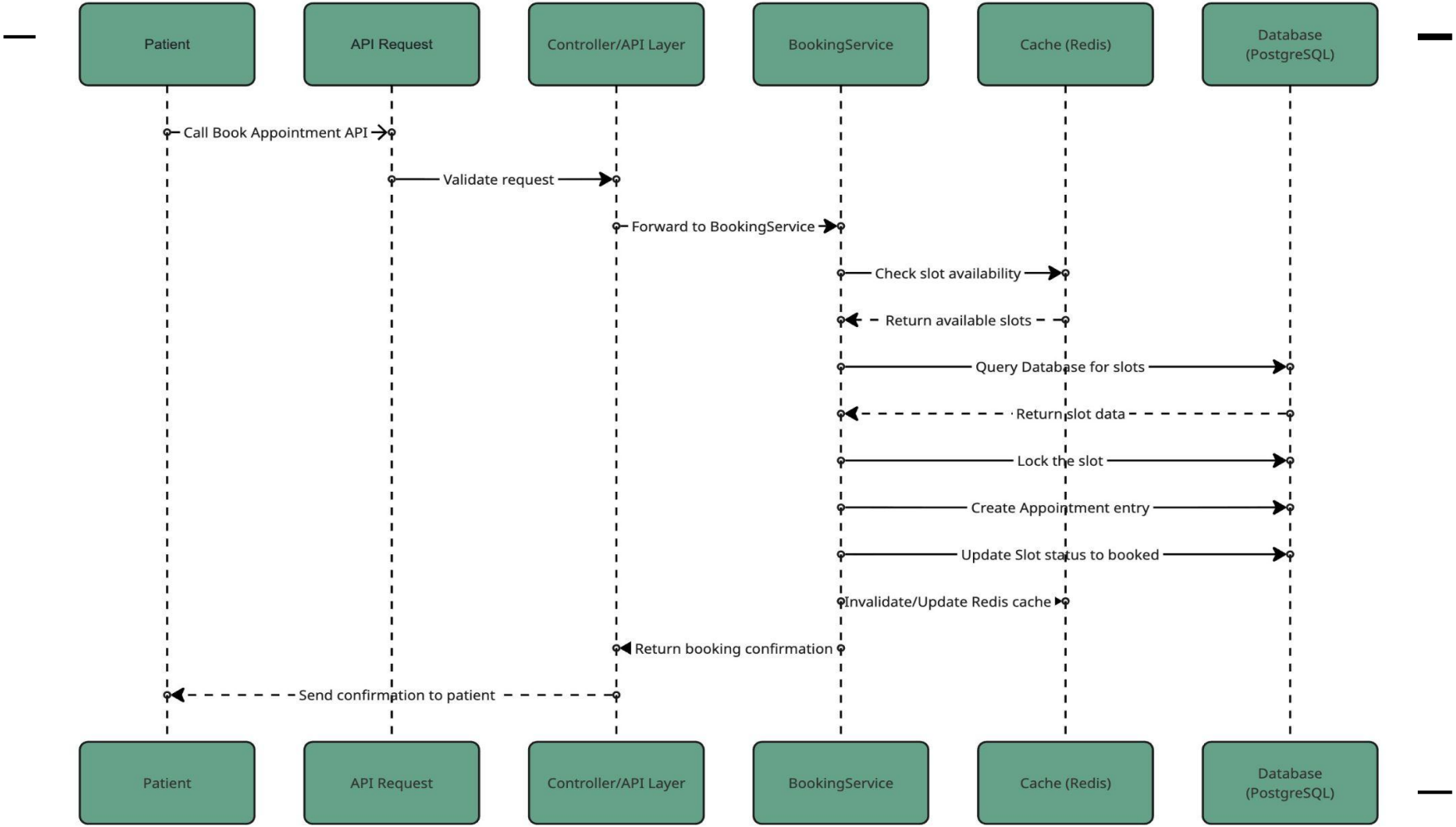

DAY_OF_WEEK
PK | id | Smallint
NOT_NULL | name | Nvarchar(20)
SCHEDULE
PK | id | Uuid
NOT_NULL | clinic_id | Uuid
NOT_NULL | doctor_id | Uuid
NOT_NULL | day_of_week | Smallint
NOT_NULL | start_time | Time
NOT_NULL | end_time | Time
SLOT
PK | id | Uuid
NOT_NULL | clinic_id | Uuid
NOT_NULL | doctor_id | Uuid
NOT_NULL | start_time | Timestamptz
NOT_NULL | end_time | Timestamptz
APPOINTMENT
PK | id | Uuid
NOT_NULL | slot_id | Uuid
NOT_NULL | patient_id | Uuid
NOT_NULL | status | Smallint
DOCTOR
PK | id | Uuid
NOT_NULL | name | Nvarchar(200)
specialization | Nvarchar(100)
CLINIC_DOCTOR_MAP
PK, FK | clinic_id | Uuid
PK, FK | doctor_id | Uuid
APPOINTMENT_STATUS
PK | id | Smallint
NOT_NULL | name | Nvarchar(20)
CLINIC
PK | id | Uuid
NOT_NULL | name | Nvarchar(200)
address | Nvarchar(500)
PATIENT
PK | id | Uuid
NOT_NULL | name | Nvarchar(200)
email | Nvarchar(200)
schedules
generates
books
has
has
maps
relates to

System Architecture

System Architecture

- **API Layer** – Handles patient requests, input validation.
- **Service Layer** – Booking & schedule logic; reads from cache/DB, writes to DB.
- **Data Layer** – PostgreSQL: source of truth for slots & appointments.
- **Cache Layer** – Redis: speeds up slot availability queries.
- **Background Worker** – Generates slots asynchronously, updates DB & cache.

Booking Flow


Patient
API Request
Controller/API Layer
BookingService
Cache (Redis)
Database (PostgreSQL)
Call Book Appointment API
Validate request
Forward to BookingService
Check slot availability
Return available slots
Query Database for slots
Return slot data
Lock the slot
Create Appointment entry
Update Slot status to booked
Invalidate/Update Redis cache
Return booking confirmation
Send confirmation to patient
Patient
API Request
Controller/API Layer
BookingService
Cache (Redis)
Database (PostgreSQL)

Scalability

Backend Scalability

- Cache the top 10-20% reads
- Use multiple server with load balancer to handle backend server spikes
- Use queue to stop DB spikes, connection pool saturation when traffic is extremely high e.g 1k+ bookings/second or more.

Database Scalability

- Don’t Try Database partitioning & Sharding unless we are hitting bottleneck
- Try proper indexing + caching first
- Then try Separate read write DBs with CQRS
- Keep Replica database for high availability and disaster recovery

Extensibilty

SMS/Email Notifications

- Separate notification service that uses 3rd-party service like Twilio, SendGrid, ...
- Event-Driven Notifications via RabbitMQ/Azure Service Bus

Online Consultations

- Integration with Zoom or others

Payment Integration

- Separate Payment module with Strategy/Factory Pattern
- Open Closed Principle

Thank you